Section 1 — Recall

Linked List Variation & Operation	Best	Average	Worst	Space	Why (Theoretical Justification)
Singly Linked List	Insert Head: $O(1)$ Delete Tail: $O(n)$	Insert Head: $O(1)$ Delete Tail: $O(n)$	Insert Head: $O(1)$ Delete Tail: $O(n)$	$O(1)$	Insert Head: Only requires updating the new node's next to head and reassigning head ($O(1)$). Delete Tail: Even with a tail pointer, a singly linked list has no backward links; updating the second-to-last node requires traversing $n - 1$ steps from head ($O(n)$).
Doubly Linked List *(Delete Given Node Pointer)*	$O(1)$	$O(1)$	$O(1)$	$O(1)$	The pointer directly exposes node->prev and node->next. Bypassing the node requires only updating prev->next = next and next->prev = prev, taking constant time without traversing.
Circular Linked List *(Insert Head with TailPtr)*	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Having a pointer directly to the tail provides immediate $O(1)$ access to both the tail and the head (tail->next). The new node is linked to tail->next, and tail->next is redirected to the new node without any traversal.
Sorted Singly Linked List *(Search &*	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Best Case: The inserted value is smaller than head->data (inserted directly at head in $O(1)$). Avg/Worst: Binary

Insert)					search is impossible because linked lists lack $O(1)$ random indexing. Finding the correct sorted slot requires sequentially traversing up to n nodes ($O(n)$).

Section 3 — Implementation: Find the Middle Node of a Singly Linked List

Complexity of Time: O(N)
Explanation : The fast pointer moves across the list twice as quickly as the slow pointer. The loop only needs to execute N/2 times to cover N elements. The temporal complexity is linear, O(N), when the constant factor is removed.
Complexity of Auxiliary Space: O(1)
Explanation: Just two extra pointer variables (slow and fast) are used by the algorithm. As a result, it does not require additional allocation.
Section 4 — Empirical & Theoretical Analysis

n	Structure / Strategy	Empirical Traversals	Theoretical Worst	Theoretical Avg	Ratio (Empirical / Avg)
10,000	Singly Linked List (Head Search)	5053.27	10,000	5,000	1.01
10,000	Doubly Linked List (Head Search)	5053.27	10,000	5,000	1.01
10,000	Doubly Linked List (Bi-directional)	2447.03	10,000	2500	0.98

Analysis

Singly Linked List: A Singly Linked List possesses only forward pointers (next) thus the search is unidimensional and starts at head. For a uniform distribution across n items, the target is equally likely to be at any rank $k \in [1, n]$, yielding an expected traversal count of $\frac{n+1}{2} \approx \frac{n}{2} = 5,000$.
Bidirectional Traversal: A Doubly Linked List provides symmetric traversal capabilities via both head->next and tail->prev thus the search space is partitioned into two symmetric halves:

$$Expectedsteps = \frac{1}{n}\left(\sum_{i=1}^{n/2} i + \sum_{i=1}^{n/2} i\right) \approx \frac{2}{n} \cdot \frac{(n/2)^2}{2} = \frac{n}{4} = 2,500$$

Section 5 — Proposed Data Structure & Theoretical Justification
Proposed Structure: Circular Doubly Linked List (CDLL)

Why Other Structures Are Insufficient

- Singly Linked List (SLL): SLL only stores a forward next pointer. It cannot navigate backward to replay the previous track (previousSong()) in $O(1)$ time.
- Jumping from the last song to the first song (or in a reversed order) either requires maintaining distinct boundary conditions or traversing the entire length of the list ($O(n)$) to reset the pointer.

Why CDLL Is Optimal

- The list forms a closed, continuous ring.
- Both forward and backward boundaries vanish. Every single node has immediate, valid $O(1)$ access to both its predecessor and successor.

Phase 1: Detailed Workflow for Track Navigation
The system uses a single pointer reference: Node* currentTrack to point at the current song playing.

1. nextSong() Operation

```
C/C++
if (currentTrack != nullptr) {
    currentTrack = currentTrack->next;
}
```

If currentTrack is at the last track, currentTrack->next naturally lands directly on head without conditional branch overhead.

2. previousSong() Operation

```
C/C++
if (currentTrack != nullptr) {
    currentTrack = currentTrack->prev;
}
```

If currentTrack is at head, currentTrack->prev moves seamlessly to tail.

Phase 2: Detailed Workflow for Playlist Management

1. Adding a Track

```C/C++
void insertAfterCurrent(Node* currentTrack, string songTitle) {
    Node* newNode = new Node(songTitle);
    if (currentTrack == nullptr) {
        newNode->next = newNode;
        newNode->prev = newNode;
        currentTrack = newNode;
        return;
    }
    Node* nextNode = currentTrack->next;
    newNode->next = nextNode;
    newNode->prev = currentTrack;
    currentTrack->next = newNode;
    nextNode->prev = newNode;
}
```

2. Deleting a Track

```C/C++
void removeTrack(Node*& currentTrack, Node* trackToDelete) {
    if (trackToDelete == nullptr) return;
    // Single-node playlist edge case
    if (trackToDelete->next == trackToDelete) {
        delete trackToDelete;
        currentTrack = nullptr;
        return;
    }
    Node* pred = trackToDelete->prev;
    Node* succ = trackToDelete->next;
    pred->next = succ;
    succ->prev = pred;
    // Advance playback if the currently playing song is deleted
    if (currentTrack == trackToDelete) {
        currentTrack = succ;
    }
```

```
    delete trackToDelete;
}
```

Infinite "Repeat All" Looping

Because the ring invariants head->prev = tail and tail->next = head are maintained across all modifications, playback never hits a nullptr exception:

When the user presses "Next" at the final song, tail->next immediately resolves to head.

When the user presses "Previous" at the opening song, head->prev immediately resolves to tail.